Clasificación de Imágenes CCTV en Entornos Portuarios

Detección de barcos y estado de atraque mediante redes convolucionales

Autor: Nicolás Aller Ponte

Asignatura: Análisis e Interpretación de Datos Audiovisuales (AIDA)

Práctica: P1 - Clasificación con CNNs

Dataset: Smartports CCTV Port Surveillance

Resumen

Este trabajo aborda dos tareas de clasificación binaria sobre imágenes de cámaras CCTV de un puerto: (1) detección de barco (*Ship/No-Ship*) y (2) estado de atraque (*Docked/Undocked*). Se comparan tres arquitecturas - una CNN ligera entrenada desde cero (SimpleCNN), una variante inspirada en ConvNeXt (MiniConvNeXt) y ResNet18 con *fine-tuning* desde ImageNet - bajo un esquema de validación cruzada de 5 iteraciones. Adicionalmente se evalúa una estrategia de *transfer learning* inter-tarea (Ship → Docked). ResNet18 domina en ambas tareas, alcanzando AUC-ROC de 0.999 en Ship y 0.861 en Docked. El transfer learning inter-tarea no parece mejorar los resultados.

1. Introducción

La automatización de la vigilancia portuaria es un caso de uso real y creciente dentro de la visión por computador aplicada a entornos industriales. Disponer de sistemas que detecten automáticamente la presencia de embarcaciones o su estado de atraque reduce la dependencia de operadores humanos y permite integrar la información en sistemas de gestión de tráfico marítimo.

En esta práctica se trabaja con el dataset *Smartports CCTV*, brindado en la Asignatura, el cual contiene imágenes de vigilancia capturadas en un puerto real. El problema se plantea como dos tareas de clasificación binaria independientes:

- **Ship/No-Ship**: ¿hay un barco visible en la imagen? (294 imágenes)
- **Docked/Undocked**: si hay barco, ¿está atracado? (184 imágenes)

El objetivo es comparar distintas estrategias de modelado, analizar el efecto del *data augmentation* y explorar si el conocimiento adquirido en la primera tarea puede transferirse a la segunda.

2. Dataset

2.1. Descripción y distribución de clases

El dataset consta de 294 imágenes JPEG en resolución variable, todas redimensionadas a 224×224 píxeles antes de ser procesadas por los modelos. Las etiquetas se distribuyen de la siguiente forma:

- **Ship/No-Ship**: 184 imágenes con barco (62.6 %) y 110 sin barco (37.4 %). El desequilibrio leve se corrige aplicando `pos_weight=0.598` en la función de pérdida `BCEWithLogitsLoss`.
- **Docked/Undocked**: 92 imágenes atracadas (50 %) y 92 no atracadas (50 %). Las clases están perfectamente balanceadas, por lo que no se aplica corrección de peso.

2.2. Partición de datos

Se emplea **validación cruzada estratificada de 5 iteraciones** (*5-Fold Stratified CV*). Dentro de cada fold, el conjunto de entrenamiento se subdivide en un 75 %/25 % de entrenamiento efectivo y validación, manteniendo la estratificación. Esto permite evaluar la variabilidad del rendimiento entre particiones y obtener estimaciones estadísticamente robustas.

El tamaño reducido del dataset (294 y 184 imágenes) justifica esta decisión: un *hold-out* simple no aportaría suficiente fiabilidad estadística.

3. Arquitecturas y Metodología

3.1. Modelos comparados

Se evalúan tres arquitecturas con niveles de complejidad y estrategias de inicialización diferentes:

SimpleCNN Red convolucional ligera diseñada desde cero. Consta de tres bloques `Conv2D` → `BN` → `ReLU` → seguidos de un clasificador con `AdaptiveAvgPool` y una capa lineal de salida. Parámetros entrenables: ∼120k. Rápido pero limitada capacidad de aprendizaje.

MiniConvNeXt Arquitectura inspirada en ConvNeXt, también entrenada desde cero. Emplea convoluciones *depthwise* 7×7, `LayerNorm` y conexiones residuales en las capas (*layer scale*). Etapas de dimensiones [40, 80, 160, 320] con profundidades [2, 2, 6, 2]. Es más compleja que SimpleCNN pero requiere más datos para generalizar, puede no generar buenos resultados con un dataset como este y así será.

ResNet18 Arquitectura preentrenada en ImageNet (`IMAGENET1K_V1`). La capa `fc` final se sustituye por una salida de dimensión 1 para clasificación binaria. Se entrena en dos fases: primero con el *backbone* congelado (épocas 1–4), y a partir de la época 5 se descongela para *fine-tuning* completo con $\mathrm{lr} = 10^{-4}$.

3.2. Configuración de entrenamiento

Hiperparámetro	**Valor**
Optimizador	Adam
LR (SimpleCNN / ConvNeXt)	10^{-3}
LR (ResNet18)	10^{-4}
Scheduler	ReduceLROnPlateau (factor 0.5, patience 3)
Función de pérdida	BCEWithLogitsLoss
Batch size	16
Épocas máximas	50
Early stopping	patience = 7 (sobre val_loss)

3.3. Data augmentation

Para evaluar su efecto se entrena cada configuración con y sin augmentation. Las transformaciones aplicadas son: volteo horizontal aleatorio, rotación (±15ř), variación de brillo/contraste y *jitter* de color. La normalización ImageNet se aplica en todos los casos.

4. Resultados: Ship / No-Ship

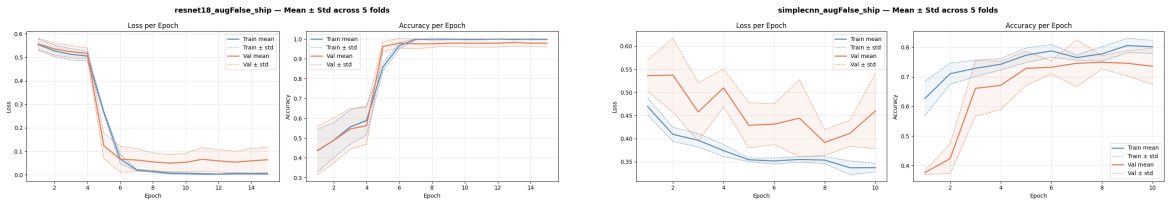
4.1. Rendimiento comparativo

Cuadro 1: Resultados en test (media ± desv. típica, 5-Fold CV) - Ship/No-Ship. Métrica principal: AUC-ROC.

Modelo	Aug	Acc	F1	AUC-ROC	AUC-PR
ResNet18	No	0.980 ± 0.013	0.984 ± 0.010	**0.999 ± 0.001**	0.999 ± 0.001
	Sí	0.973 ± 0.023	0.979 ± 0.017	**0.999 ± 0.001**	0.999 ± 0.001
SimpleCNN	No	0.846 ± 0.020	0.882 ± 0.016	0.891 ± 0.024	0.918 ± 0.032
	Sí	0.772 ± 0.058	0.826 ± 0.032	0.814 ± 0.039	0.862 ± 0.047
MiniConvNeXt	No	0.779 ± 0.092	0.827 ± 0.062	0.836 ± 0.088	0.898 ± 0.059
	Sí	0.697 ± 0.078	0.754 ± 0.072	0.765 ± 0.052	0.837 ± 0.025

4.2. Análisis

La tarea de detección de barcos resulta **relativamente sencilla para ResNet18**, que alcanza un AUC-ROC prácticamente perfecto (0.999) gracias a los features de bajo nivel aprendidos en ImageNet. La diferencia con los modelos entrenados desde cero es notable: SimpleCNN logra un AUC-ROC de 0.891, razonable para una red tan ligera, mientras que MiniConvNeXt sorprende con resultados inferiores a pesar de ser más compleja, como indicamos antes su complejidad complica su capacidad de entrenamiento en un dataset de estas caracteristicas (∼230 imágenes de entrenamiento efectivas).



(a) Historial de entrenamiento - ResNet18 (sin aug, ship). El descongelado del *backbone* en época 5 produce una caída pronunciada de la pérdida de validación.

(b) Historial de entrenamiento - SimpleCNN (sin aug, ship). Convergencia más lenta y algo inestable a lo largo de los 50 épocas.

Figura 1: Curvas de pérdida y precisión (media ± desv. típica entre folds) para los dos modelos más representativos en la tarea Ship/No-Ship.

Respecto al *data augmentation*, su efecto es neutro o ligeramente negativo en la tarea Ship. Esto es coherente ya que las imágenes de barcos tienen patrones visuales suficientemente distintos de las imágenes sin barco (la presencia de este), por lo que los modelos generalizan bien sin necesidad de aumentar artificialmente la variabilidad.

5. Resultados: Docked / Undocked

5.1. Rendimiento comparativo

Cuadro 2: Resultados en test (media ± desv. típica, 5-Fold CV) - Docked/Undocked.

Modelo	**Aug**	**Acc**	**F1**	**AUC-ROC**	**AUC-PR**
ResNet18	No	0.831 ± 0.069	0.804 ± 0.106	**0.861 ± 0.055**	0.877 ± 0.053
	Sí	0.771 ± 0.120	0.738 ± 0.203	0.855 ± 0.068	0.882 ± 0.056
SimpleCNN	No	0.690 ± 0.102	0.648 ± 0.137	0.752 ± 0.117	0.786 ± 0.112
	Sí	0.597 ± 0.053	0.588 ± 0.141	0.660 ± 0.026	0.677 ± 0.027
MiniConvNeXt	No	0.674 ± 0.088	0.725 ± 0.252	0.700 ± 0.118	0.719 ± 0.105
	Sí	0.635 ± 0.075	0.643 ± 0.096	0.639 ± 0.069	0.670 ± 0.093

5.2. Análisis

La tarea Docked/Undocked es **notablemente más difícil**. La diferencia entre un barco atracado y uno navegando próximo al muelle puede reducirse a detalles espaciales sutiles - posición relativa con respecto a la estructura del puerto, presencia de cabos, etc. - que requieren features de mayor nivel semántico.

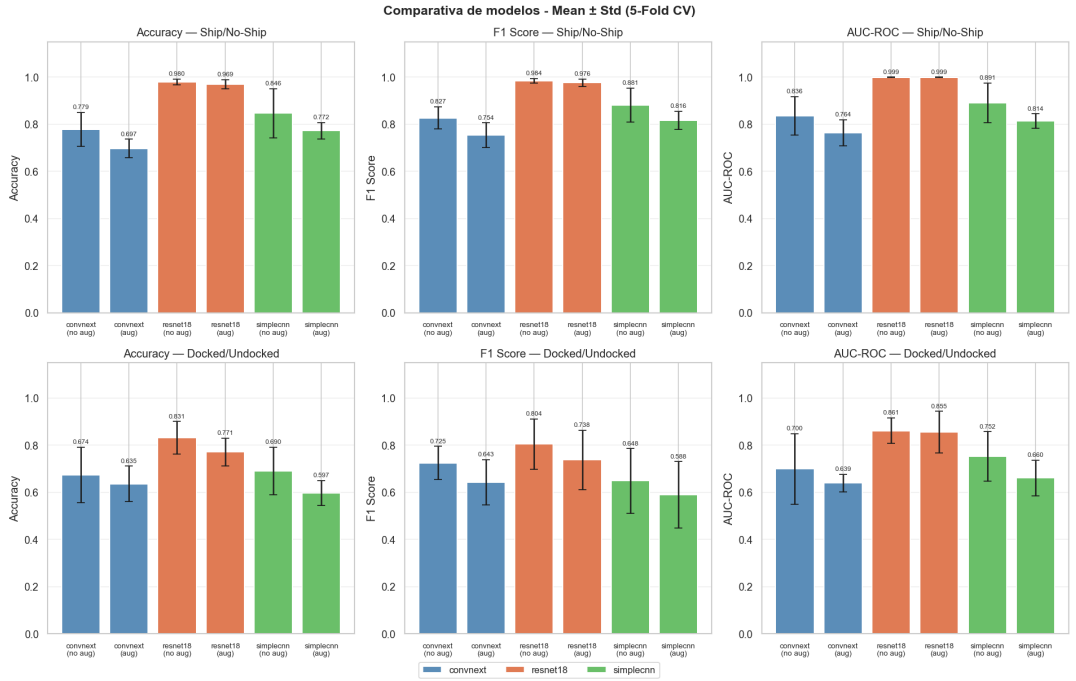


Figura 2: Comparativa de todos los modelos para ambas tareas (Accuracy, F1 y AUC-ROC). Las barras de error representan la desviación típica entre folds.

ResNet18 mantiene su ventaja con un AUC-ROC de 0.861, pero la brecha con los modelos entrenados desde cero es menor que en Ship. La varianza entre folds es mayor en todos los modelos, lo que refleja la mayor dificultad intrínseca de la tarea y el tamaño más reducido del dataset (184 imágenes).

El *data augmentation* empeora los resultados en esta tarea, especialmente para SimpleCNN. Con tan pocas muestras, las transformaciones pueden introducir variabilidad artificial que dificulta aprender las relaciones espaciales clave para distinguir el estado de atraque.

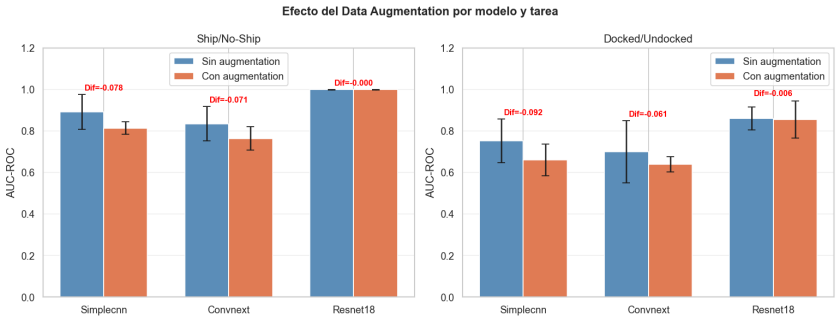


Figura 3: Efecto del *data augmentation* sobre el AUC-ROC en ambas tareas. En Ship el efecto es neutro; en Docked es sistemáticamente negativo.

6. Experimento de Transfer Learning Inter-tarea

6.1. Motivación y diseño

Dado el rendimiento moderado en la tarea Docked/Undocked, se plantea una hipótesis: ¿puede el conocimiento adquirido durante el entrenamiento en Ship/No-Ship mejorar el rendimiento en Docked/Undocked?

La intuición es que los modelos entrenados en Ship han aprendido a reconocer la silueta y estructura visual de las embarcaciones - información que podría ser directamente útil para determinar el estado de atraque. Esta estrategia es especialmente relevante para SimpleCNN, que parte de pesos aleatorios y se beneficiaría de una inicialización más informativa.

El procedimiento es el siguiente para cada fold k:

1. Cargar el checkpoint del modelo entrenado en Ship (fold k).
2. Reinicializar la capa de clasificación final con pesos Xavier.
3. Descongelar todos los parámetros y fine-tunear sobre el conjunto Docked con $\text{lr} = 10^{-4}$.

6.2. Resultados y discusión

Cuadro 3: Comparativa transfer learning vs. entrenamiento desde cero en la tarea Docked/Undocked (AUC-ROC, media ± desv. típica).

Modelo	**Aug**	**Desde cero**	**Transfer**	**Δ**
SimpleCNN	No	0.752 ± 0.117	0.736 ± 0.131	−0.016
	Sí	0.660 ± 0.026	0.656 ± 0.103	−0.004
ResNet18	No	0.861 ± 0.055	0.841 ± 0.067	−0.020
	Sí	0.855 ± 0.068	0.860 ± 0.065	+0.005

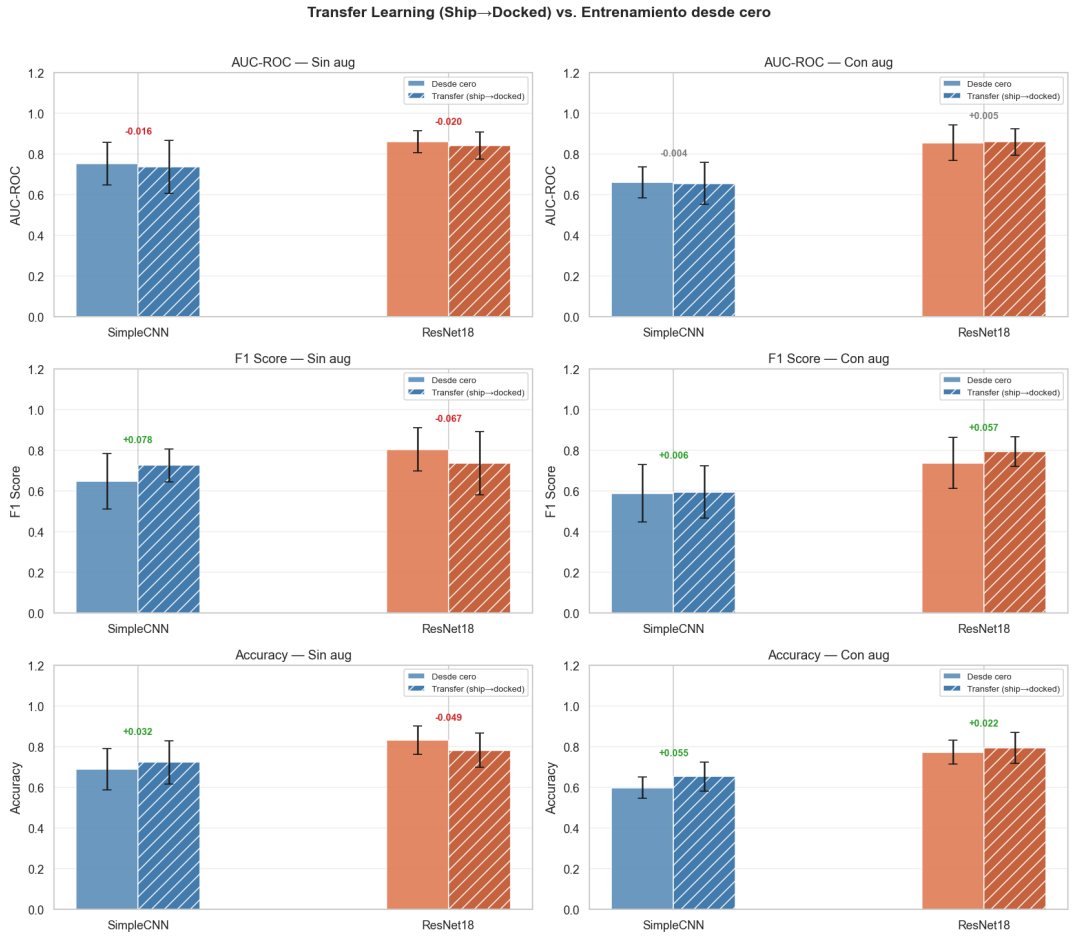


Figura 4: Comparativa visual entre entrenamiento desde cero y transfer learning (Ship → Docked) para tres métricas y dos configuraciones de augmentation. El delta sobre cada par indica la diferencia absoluta.

La hipótesis **no se confirma**: el transfer learning inter-tarea no mejora los resultados base de forma consistente. Los deltas son pequeños y mayoritariamente negativos.

Esto puede explicarse por dos motivos complementarios:

- **Para ResNet18**: el modelo ya parte de ImageNet, que proporciona features visuales de muy alta calidad. El entrenamiento en Ship/No-Ship no añade información relevante que ImageNet no proporcionara ya.
- **Para SimpleCNN**: las features aprendidas para detectar la *presencia* de un barco (bordes, silueta global) no son las mismas que las necesarias para determinar su *posición relativa* respecto al muelle. La tarea Docked/Undocked exige razonar sobre la relación espacial entre el barco y la infraestructura portuaria, lo que requiere features de mayor nivel semántico que las aprendidas en Ship.